

Conditionals

CSCI 1101

2024-02-08

Contingency

Sometimes we want to do something *contingent* on some condition:

If it's raining then take an umbrella.

In python we can do this with a **conditional statement** (a.k.a. “if-statement”).

```
if <guard> :  
    <branch>
```

The **guard** is any expression or function call that results in a boolean value.

The **branch** is any nonempty sequence of statements or function calls, called a **block**, that is indented by one level from the guard.

Conditional Execution (one branch)

When python encounters a conditional statement the following happens:

- The guard is evaluated to a boolean value (a.k.a. “tested”).
- If the guard evaluates to `True` (a.k.a. “is satisfied”) then the branch is **taken**:
 - control passes to the first branch statement,
 - control flows within the branch until the last branch statement,
 - after the last branch statement control passes to the first statement following the conditional statement.
- If the guard evaluates to `False` then the branch is **skipped**:
 - control passes directly to the first statement following the conditional statement.

Favorite Subject

```
def fav_subject() :  
    fav = input("What's your favorite subject? ")  
    if str.lower(fav) == "computer science" :  
        print("That's my favorite too!")  
    print(str.title(fav) + " is a great subject.")
```

Multiple Branches

A conditional statement may have multiple branches.

The first branch always starts with the keyword `if`.

Subsequent branches start with the keyword `elif` (short for “else if”).

You can have as many `elif` branches as you like.

```
if <guard_0> :  
    <branch_0>  
elif <guard_1> :  
    <branch_1>  
    :  
elif <guard_n> :  
    <branch_n>
```

Conditional Execution (multiple branches)

When python encounters a multi-branch conditional statement the following happens:

- The first guard is *tested*.
- If the first guard is *satisfied* then the first branch is *taken* and all subsequent branches are *skipped*.
- otherwise, the first branch is *skipped* and the next guard is *tested*.
- If that guard is *satisfied* then its branch is *taken* and all subsequent branches are *skipped*.
- This process continues until either some branch is *taken*, or else the last guard evaluates to `False` and all branches are *skipped*.

Note that *at most one* branch is taken.

Animal, Vegetable, or Mineral

```
def guessing_game () :  
    print("Think of a thing.")  
    if input("Is it an animal? ") == "yes" :  
        print("Then it must be a capybara")  
    elif input("Is it a vegetable? ") == "yes" :  
        print("Then it must be kale")  
    elif input("Is it a mineral? ") == "yes" :  
        print("Then it must be pyrite")
```

Default Branch

Sometimes we want to ensure that *exactly one* branch is taken.

We can do this by making the guard of the last (why?) branch `True`.

This is called the **default branch**.

There is a shorthand for this: the keyword `else` means `elif True`.

```
if <guard_0> :  
    <branch_0>  
elif <guard_1> :  
    <branch_1>  
    :  
elif <guard_n> :  
    <branch_n>  
else :  
    <default_branch>
```


Activity: CS Survey

Write a function `cs_survey : () --> NoneType` which asks the user if they like computer science and if their answer (ignoring case) is:

`yes` then agrees that c.s. is great,

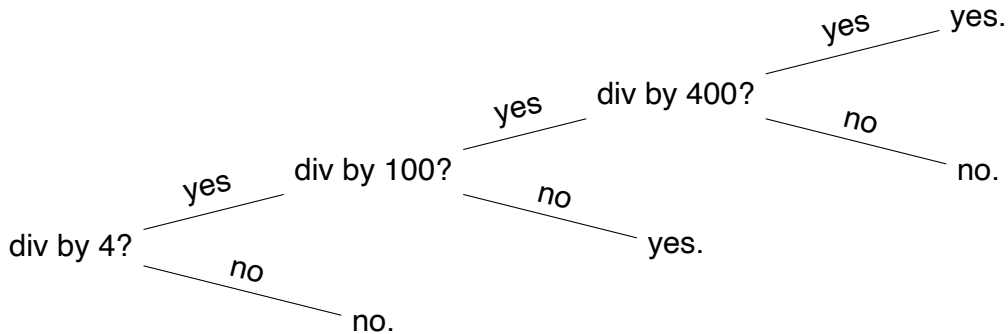
`no` then politely disagrees,

`anything else` then apologizes for not understanding their answer.

Complex Conditionals

Complex conditional structure can get confusing.

How do we know if it's a leap year?



Complex Conditionals

Complex conditional structure can get confusing.

How do we know if it's a leap year?

```
def leap_deep(y) :  
    if y % 4 == 0 :  
        if y % 100 == 0 :  
            if y % 400 == 0 :  
                print(str(y) + " is a leap year")  
            else :  
                print(str(y) + " is not a leap year")  
        else :  
            print(str(y) + " is a leap year")  
    else :  
        print(str(y) + " is not a leap year")
```

Boolean Expressions

It's better to use boolean expressions in branch guards to reduce this complexity.

For deciding leap years we need only two branches:
one for leap years, and one for non-leap years.

```
def leap_flat(y) :  
    if (y % 4 == 0 and y % 100 != 0) or y % 400 == 0 :  
        print (str(y) + " is a leap year")  
    else :  
        print (str(y) + " is not a leap year")
```

Activity: What to Do?

Write a function `what_to_do : (str) --> str`
that returns:

- "study" if the argument (ignoring case) is "Sunday", "Monday", or "Tuesday",
- "party" if the argument (ignoring case) is "Thursday", "Friday", or "Saturday",
- "dunno" if the argument is anything else.

Try to use the fewest cases possible.

Conditional Expressions

The conditionals we have seen so far are **statements**: the branches are *blocks* that can be run but don't evaluate to values and can't be used in expressions.

Python also had a **conditional expression**:

```
<true_expression> if <guard> else <false_expression>
```

If the guard evaluates to `True` then the conditional expression evaluates to its `true_expression`.

If the guard evaluates to `False` then the conditional expression evaluates to its `false_expression`.

A conditional expression can be used anywhere an expression is allowed, including within other expressions.

Reducing Duplication with Conditional Expressions

Conditional expressions can be used to reduce duplication of code.

Compare:

```
def min_s(num_1 , num_2) :  
    if num_1 < num_2 :  
        return num_1  
    else :  
        return num_2
```

with:

```
def min_e(num_1 , num_2) :  
    return num_1 if num_1 < num_2 else num_2
```

Activity: Conditional Expressions

Use a conditional expression to write a function

```
number_size : () --> NoneType
```

that asks the user to enter a number, and remarks that it's either big or small.

For example:

```
> number_size()
```

```
Please enter a number: 7
```

```
That's pretty small!
```

```
> number_size()
```

```
Please enter a number: 42
```

```
That's pretty big!
```

Your function should call `print` only *once*.

To Do

Finish *Homework 3: Conditionals* on CodeRunner by 11:00AM next Thursday.